

QB: Chapter 1.3 — Pointers, Records & Advanced Arrays

Unit 1 | 4×2 -mark | 5×5 -mark | Total: 9 questions Lectures: L13–L15 |
BT Levels: BT1, BT2, BT3 | COs: CO2, CO4

Section A: 2-Mark Questions

Q1. [BT1 | CO2] Define a *pointer*. Distinguish between: (a) a pointer variable, (b) the address it holds, and (c) the value at that address (dereferenced value). Use a short C-style notation example to illustrate all three.

Q2. [BT1 | CO2] Define a *sparse matrix*. State a commonly used threshold for when a matrix is considered sparse. Give two real-world examples of systems that produce sparse matrices.

Q3. [BT2 | CO2] Explain *memory alignment padding* in C structs. Why does the compiler insert padding bytes? Provide a concise example showing a struct that requires padding and state its actual size in bytes.

Q4. [BT2 | CO2] Describe the *triplet (COO) representation* of a sparse matrix. State the three values stored per non-zero element. What additional metadata must be stored alongside the triplet list?

Section B: 5-Mark Questions

Q5. [BT3 | CO2] In C, given `int arr[5] = {10, 20, 30, 40, 50}` and `int *p = arr`:

Evaluate the following expressions and show your reasoning: (a) `*(p + 2)` (b) `p[3]` (c) `*(arr + 1)` (d) `p[0] + *(p + 4)`

Explain in terms of byte offsets how pointer arithmetic advances through the array when incrementing by 1 on an `int * pointer` (assume `int = 4` bytes, base address = 1000).

Q6. [BT3 | CO2] Consider the following struct (`char = 1 B`, `int = 4 B`, `double = 8 B`):

```
struct Employee {
    char    id[5];
    double  salary;
    int     dept;
    char    grade;
};
```

(a) Compute the size of this struct as declared, showing all padding bytes inserted by the

compiler for alignment. (b) Suggest a rearrangement of fields that minimises padding. Compute the new size and show the memory layout with offsets.

Q7. [BT3 | CO2] Represent the following 5×5 sparse matrix in *triplet (COO) format*:

```
[ 0  0  6  0  0 ]
[ 0  0  0  0  0 ]
[ 3  0  0  0  8 ]
[ 0  0  0  0  0 ]
[ 0  4  0  0  0 ]
```

(a) List all triplets (row, col, value). (b) Calculate the memory used by the dense representation and by the COO representation (int = 4 bytes). Compute the percentage memory saving. (c) State the time complexity of element lookup in COO format.

Q8. [BT3 | CO2] Define a *pointer array* (array of pointers). Implement one in Python that stores references to four student name strings. Demonstrate that modifying the list element via the pointer array changes the accessible data. Also implement an array of pointers to five integer values and show how iterating through it differs from a regular integer array.

Q9. [BT3 | CO2, CO4] Explain the *Compressed Sparse Row (CSR)* representation for sparse matrices. It uses three arrays: `values`, `col_ind`, and `row_ptr`.

(a) Convert the sparse matrix from Q7 into CSR format. Show all three arrays explicitly. (b) Describe how to retrieve element at position (2, 4) using the CSR structure. Show the index arithmetic. (c) Compare COO and CSR in terms of: storage space (in bytes for the Q7 matrix), element lookup time complexity, and suitability for matrix-vector multiplication.